Sequential Influence Maximization in a Workplace Contact Network

SE4095 Social Network Theory and Practice - Assignment 03
Independent Cascade, REINFORCE, and an MLP versus Transformer policy

An organization can promote an awareness message to only five participants. We treat seed selection as a five-step sequential decision problem on the supplied 92-participant SocioPatterns workplace graph. The fixed graph has 755 edges, density 0.1804, one connected component, and average clustering 0.4260. Independent Cascade is implemented with activation probability 0.10 and one reproducible random draw per directed arc. We train an MLP policy and a compulsory Transformer policy with REINFORCE, using 1,000 episodes and 3 seeds per architecture. All methods are evaluated on the same 100 diffusion realisations (seeds 900000-900099). On mean spread, Degree is 59.67, Random is 60.24, the MLP is 57.53, and the Transformer is 58.86. We report the result without claiming that a learned policy must beat Degree, and we limit all conclusions to this fixed aggregated graph and simulator.

1. Network analysis

The supplied package contains 92 participants and 755 distinct undirected contacts, aggregated over the full observation period. The graph is unweighted, has no self-loops or duplicate pairs, and every stored edge satisfies source < target and is sorted. Density is 2E/[N(N-1)] = 2(755)/[92(91)] = 0.180363. All 92 nodes form one connected component; there are no isolates. The mean degree is 16.41, the minimum degree is 4, and the maximum degree is 44. Department labels are used only for visualization and descriptive analysis; they are not features, rewards, or tuning signals.

Statistic	Value
Participants	92
Undirected edges	755
Density	0.180363
Connected components	1
Isolated participants	0
Mean degree	16.41
Average clustering coefficient	0.4260

Node	Orig.	Dept	Degree	Deg. cent.	Clustering
84	804	DMCT	44	0.4835	0.2558
58	311	DSE	38	0.4176	0.2831
14	95	DSE	32	0.3516	0.3427
12	80	DMCT	31	0.3407	0.3054
27	134	DMCT	30	0.3297	0.2897
9	63	SRH	28	0.3077	0.2513
45	223	DSE	28	0.3077	0.3360
24	123	DSE	26	0.2857	0.2708
72	662	DSE	26	0.2857	0.4031
86	826	DISQ	26	0.2857	0.3231

Table 1. Network statistics and the ten participants with the largest degree. Degree centrality is degree/(N-1); ties in the ranking are broken by smaller node ID.

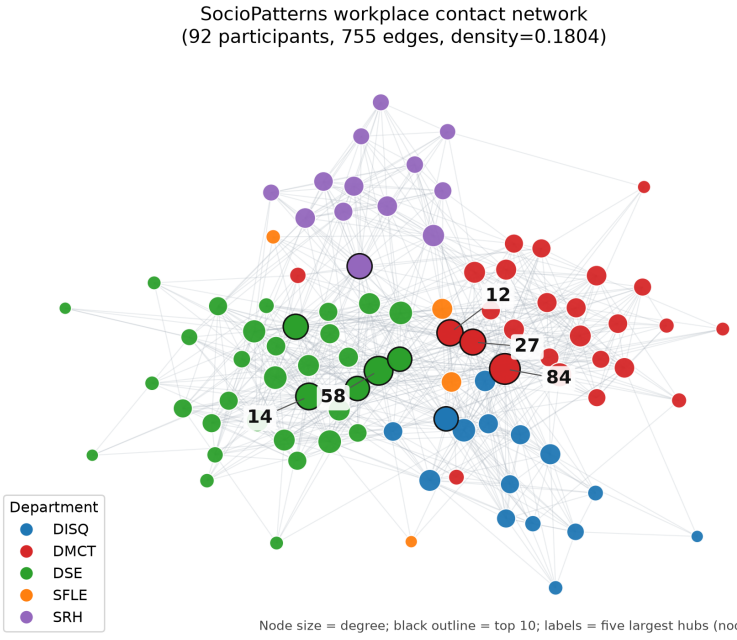


Figure 1. Workplace network; node size = degree, colour = department, black outlines = ten largest degrees, labels = five largest hub IDs (original IDs are in Table 1).

2. Reinforcement-learning formulation

An episode is a finite-horizon Markov decision process with exactly five selections. The observation is a 92 x 5 matrix in ascending participant-ID order: (i) degree centrality, degree/(N-1); (ii) the local clustering coefficient on the fixed graph; (iii) a selected indicator; (iv) the fraction of a participant's neighbours that are already selected, defined as 0 for an isolate; and (v) remaining budget divided by 5. Static features (i) and (ii) are computed once. The selection features (iii)-(v) are refreshed after every action. The action is an integer 0-91 identifying an unselected participant. The Boolean mask is True for available participants and False after selection. Invalid actions, including duplicates, raise an error rather than being silently replaced.

The first four rewards are exactly zero. Diffusion is executed only after the fifth action, and the fifth reward is final_spread/92, where the final spread includes the five seeds. The episode then terminates; the total return is in [0,1]. The task is sequential because each choice changes the remaining budget and the selected-neighbour features, so the value of a participant depends on the other four choices. There is no reward shaping, penalty, or intermediate cascade feedback.

3. Independent Cascade simulator

Each undirected edge is represented by two directed arcs, and the complete arc list is sorted by (source, target). For diffusion seed d, numpy.random.default_rng(d) draws exactly one uniform number for every arc in that order. An arc is live when its draw is below 0.10. Starting from the five seeds, breadth-first reachability over live arcs gives the final active set; active users stay active and the seeds are always counted. The sampled draws and live arcs are private attributes of the simulator. Policies receive only the five observable features and never inspect the random state or query hypothetical outcomes.

Automated verification covered an empty seed set, probability 0, probability 1, and exact repeatability for the same graph, seed set, and diffusion seed. Probability 0 and 1 were used only for tests; every training and evaluation episode used probability 0.10. Reusing the same ordered arc list keeps the realisation identical across methods for a given diffusion seed. The generated checks.json records 13/13 passing automated checks.

4. REINFORCE implementation

Training collects 20 complete episodes before each update. Actions are sampled from the masked categorical policy, and the five differentiable log-probabilities are retained. For episode i with terminal reward R_i and running baseline b, the loss is L(theta) = -(1/20) * sum_i [(R_i - b) * sum_t log pi_theta($a_{i,t}$ | $s_{i,t}$)]. Because the discount factor is 1.0, the same terminal return is used at all five selections. Rewards and the baseline are plain scalars, so they carry no gradient. The baseline starts at 0 and is held fixed while a batch is collected and its loss is computed; after the optimizer step it is updated as b = 0.9b + 0.1 * batch_mean_reward. The trajectories are then discarded.

Adam uses learning rate 0.001. One update is performed per 20-episode batch, giving exactly 50 updates per 1,000-episode run. The gradient norm is clipped to 1.0, and no PPO, critic, replay buffer, entropy bonus, advantage normalisation, reward normalisation, schedule, or early stopping is used. Python, NumPy, and PyTorch are seeded with the run seed (7, 21, or 42). Episode e uses diffusion_seed = 1,000,000 + 10,000 * train_seed + e. Training and evaluation share the graph; only the diffusion realisation changes.

5. Policy architectures

MLP. The same 5-64-64-1 network is applied independently to every participant: Linear(5,64) -> ReLU -> Linear(64,64) -> ReLU -> Linear(64,1). It returns 92 logits. Selected participants are masked and softmax is taken over the remaining participants. The MLP has no dropout, no extra layers, and no communication between participants; sequential information enters only through the five features.

Transformer. One participant is one token. A Linear(5,64) projection is followed by one standard PyTorch post-layer-normalisation encoder layer (d_model=64, 4 heads, feed-forward dimension 128, ReLU, dropout 0), then Linear(64,1) on each token. There are no positional embeddings, node-ID embeddings, causal masks, or extra final encoder normalisation. Attention is unrestricted across all 92 tokens, and selected tokens remain visible to self-attention; they are hidden only at the final action-selection stage. In each layer, every token forms queries, keys, and values; four attention heads learn different pairwise weightings, and residual connections plus normalization make the layer trainable. Attention is a learned all-to-all interaction, not the contact graph. The MLP scores each participant from its own row, whereas the Transformer can condition a participant's logit on the current selection pattern of all other participants.

6. Controlled evaluation

Random samples five distinct participants uniformly with a separate NumPy generator seeded by diffusion_seed + 10,000,000. Degree selects the five highest-degree participants, with ties broken by smaller node ID. The six final checkpoints act greedily under the mask; exact probability ties also go to the smaller ID, gradients are disabled, and no parameter is updated. Every method is evaluated on diffusion seeds 900000-900099, giving 800 episodes: 6 checkpoints x 100, Random x 100, and Degree x 100. Selection time excludes diffusion; for neural policies it measures observation-to-logit computation, masking, and argmax, while Random's sampling is included. Training time includes all training simulations and was measured with a monotonic performance counter on CPU.

7. Results

Table 2 reports every method/training-seed combination from the 800 evaluation episodes. The standard deviations are population standard deviations over the 100 shared diffusion realisations. Table 3 aggregates the three run-level means of each architecture.

Method	Seed	Mean spread	Pop. SD	Mean reward	Selection ms	Training s
Random	-1	60.24	9.72	0.6548	0.11	0.0
Degree	-1	59.67	7.26	0.6486	0.00	0.0
REINFORCE MLP	7	58.88	13.72	0.6400	1.70	8.4
REINFORCE MLP	21	58.09	16.11	0.6314	2.20	8.3
REINFORCE MLP	42	55.62	18.37	0.6046	2.32	8.5
REINFORCE Transformer	7	58.09	16.11	0.6314	4.31	23.5
REINFORCE Transformer	21	59.67	7.26	0.6486	4.30	21.8
REINFORCE Transformer	42	58.81	14.74	0.6392	4.26	20.0

Table 2. Evaluation summary. Random and Degree use train_seed = -1 and training_seconds = 0.

Architecture	Runs	Mean of run means	Pop. SD of run means
REINFORCE MLP	3	57.530	1.389
REINFORCE Transformer	3	58.857	0.646

Table 3. Run-level aggregate for each neural architecture.

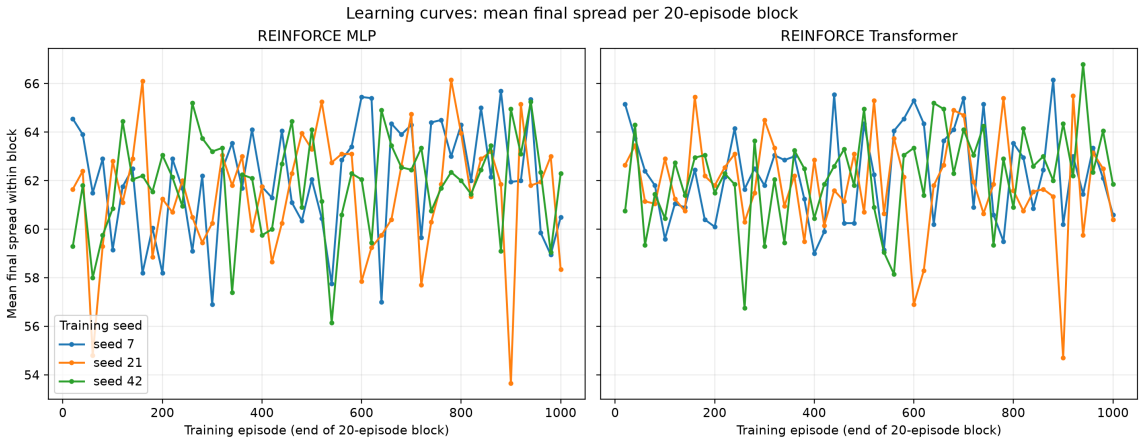


Figure 2. Learning curves for all six runs. Each point is the mean final spread in a non-overlapping block of 20 training episodes.

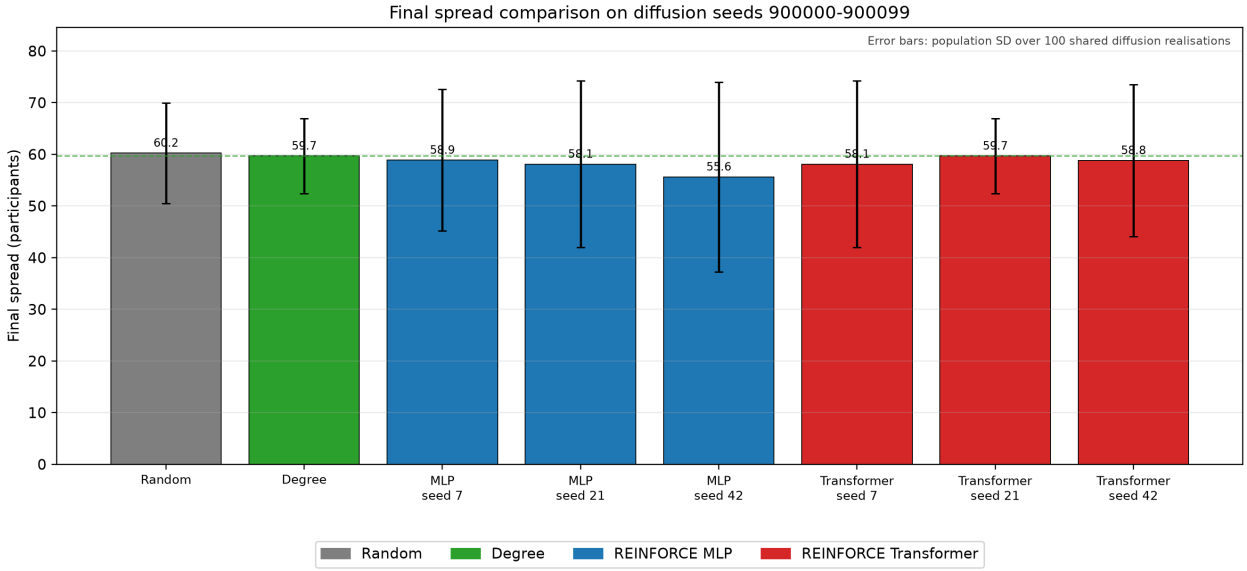


Figure 3. Final spread on the common 100 diffusion realisations. Bars show means; error bars are population standard deviations.

Method	Seed	Selected node IDs	Departments (count)	Coverage
Degree	-1	84, 58, 14, 12, 27	DMCT x3, DSE x2	2
REINFORCE MLP	7	66, 0, 36, 82, 85	DISQ x1, DSE x3, SRH x1	3
REINFORCE MLP	21	0, 36, 66, 82, 10	DISQ x1, DMCT x1, DSE x2, SRH x1	4
REINFORCE MLP	42	13, 71, 26, 89, 17	DMCT x2, DSE x2, SFLE x1	3
REINFORCE Transformer	7	66, 0, 36, 82, 10	DISQ x1, DMCT x1, DSE x2, SRH x1	4
REINFORCE Transformer	21	84, 58, 14, 12, 27	DMCT x3, DSE x2	2
REINFORCE Transformer	42	0, 36, 66, 82, 64	DISQ x1, DSE x3, SRH x1	3

Table 4. Selected participants and department coverage for the deterministic selections. Department counts are shown for the five selected users; coverage is the number of distinct departments represented. Random's mean coverage over its 100 episodes is 3.09 (population SD 0.69, range 2-4). Coverage is descriptive only.

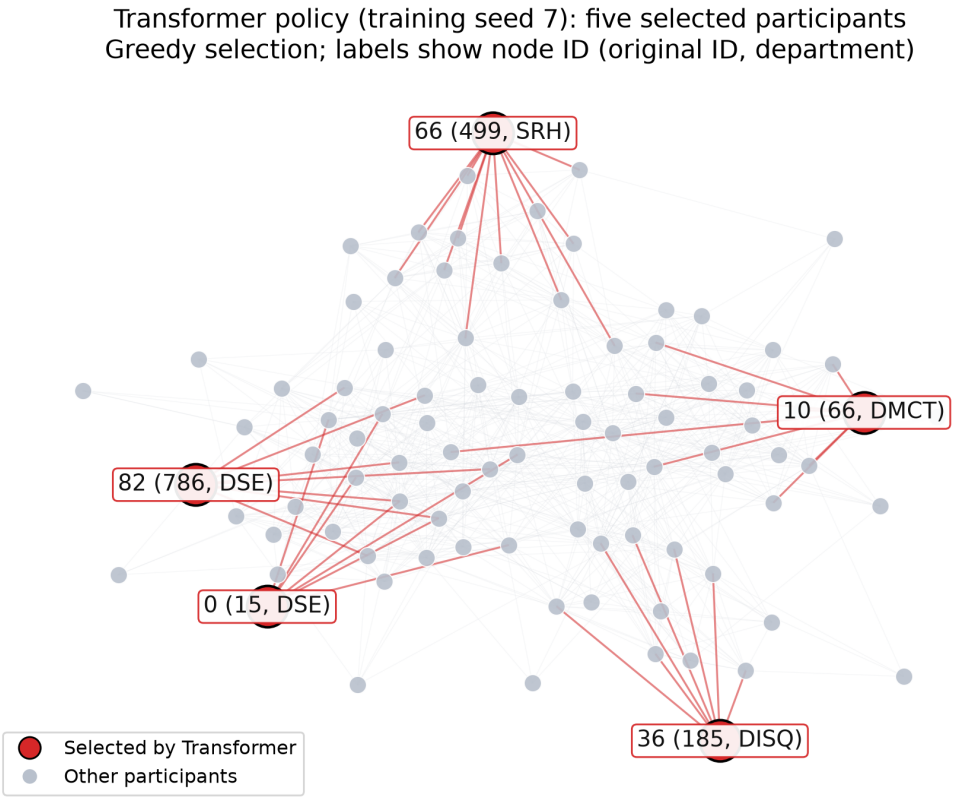


Figure 4. The five participants selected by the Transformer checkpoint trained with seed 7. The selection is identical across diffusion realisations because the policy sees only the fixed graph and the evolving selection state.

8. Discussion

Comparison with baselines. Degree's mean spread is 59.67, versus 60.24 for Random. The MLP run means are 58.88, 58.09, 55.62 and the Transformer run means are 58.09, 59.67, 58.81. Neither learned architecture exceeds Degree's mean (59.67); the gap for the stronger architecture is 0.81 participants. The comparison is descriptive: it uses the same 100 realisations for every method, but no significance test or tuning on evaluation results was performed.

Overlapping audiences and department coverage. Internal edges among the selected sets quantify overlap: Degree: 8 internal edges, 75 distinct neighbours; REINFORCE MLP seed 7: 0 internal edges, 39 distinct neighbours; REINFORCE MLP seed 21: 0 internal edges, 38 distinct neighbours; REINFORCE MLP seed 42: 1 internal edge, 33 distinct neighbours; REINFORCE Transformer seed 7: 0 internal edges, 38 distinct neighbours; REINFORCE Transformer seed 21: 8 internal edges, 75 distinct neighbours; REINFORCE Transformer seed 42: 1 internal edge, 35 distinct neighbours. Degree's five hubs are therefore not five independent audiences. Department coverage varies across selections; the full selected-user table and the Random coverage distribution are saved in results/selected_users.csv and results/department_coverage.csv. These labels remain descriptive and never enter a policy, reward, or tuning decision.

Learning stability and cost. The MLP run-level mean spread has SD 1.39; the Transformer's is 0.65. The learning curves show how the 20-episode block means move during training. On CPU, training took 8.4 s per MLP run and 21.8 s per Transformer run, including simulations; greedy selection averaged 2.07 ms per MLP episode and 4.29 ms per Transformer episode. Degree selection is effectively instantaneous (0.0007 ms). Wall-clock values are specific to this machine and run.

MLP versus Transformer. The MLP is a per-participant scorer, so its only context is the five features of that participant. The Transformer can represent interactions among all 92 participant tokens, including which users were already selected, after every action. With only 50 updates and a terminal scalar reward, that extra capacity is not guaranteed to help; the observed

run-level means and their spread are the relevant evidence, not architecture size alone.

9. Limitations

- The graph is a static aggregation of the supplied contacts. Contact timing, duration, and repeated interactions are discarded, so the analysis cannot represent when a message would spread.
- Independent Cascade at probability 0.10 is a simulation assumption, not observed message adoption. Final spread is a model quantity and not a measured outcome.
- The study uses one fixed 92-participant graph, three training seeds, and 100 evaluation realisations. The results may not transfer to other networks or contact periods.
- No hyperparameter search, early stopping, or evaluation-based tuning was used. The reported learning behaviour is therefore a controlled baseline rather than an optimised system.
- Department labels describe recorded organizational units; they are not used as inputs or objectives, and coverage is not a causal outcome.
- Timing is CPU- and machine-specific. Selection timing excludes diffusion, but small timing differences between Python-level operations should not be over-interpreted.
- The learned policies are compared with Degree and Random on the same realisations; the assignment does not require a learned policy to outperform Degree.

10. Reproducibility and generated artifacts

The complete pipeline is executed from the project root with `python main.py --output results`. The supplied CSV files are checked against their published SHA-256 hashes before use and remain unchanged. Python, NumPy, and PyTorch are seeded per run; diffusion seeds follow the prescribed formula; each checkpoint stores its architecture settings and training configuration. The checks are executed assertions, not manually entered booleans. The recorded environment was Python 3.12.10, PyTorch 2.15.0+cpu, NumPy 2.5.3, pandas 2.3.3, and NetworkX 3.6.1 on 8 logical CPU cores with Windows-11-10.0.26300-SP0.

Artifact	Observed content
training.csv	6,000 rows: 2 architectures x 3 seeds x 1,000 episodes
evaluation.csv	800 rows: 8 method/training-seed groups x 100 shared seeds
summary.csv	8 rows, one per method/training-seed combination
checkpoints/	6 final model checkpoints with architecture metadata
checks.json	13/13 checks passed in this run
figures/	Network analysis, learning curves, comparison, and Transformer selection
Pipeline time	101.1 s for training, evaluation, figures, and checks before the PDF build (machine-specific)

Table 5. Reproducibility evidence for the reported run.

References

SocioPatterns. Contacts in a Workplace. Data collected 24 June to 3 July 2013. https://sociopatterns.org/datasets/contacts-in-a-workplace/ (accessed 2026).

M. Génois, C. L. Vestergaard, J. Fournet, A. Panisson, I. Bonmarin, and A. Barrat. Data on face-to-face contacts in an office building suggest a low-cost vaccination strategy based on community linkers. Network Science, 3(3):326-347, 2015. https://doi.org/10.1017/nws.2015.10

R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8:229-256, 1992. https://doi.org/10.1007/BF00992696

A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems 32, 2019.

A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference, 2008.

J. D. Hunter. Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3):90-95, 2007.

Implementation note: all assignment code was written for this submission. PyTorch's standard nn.TransformerEncoderLayer was used as permitted; no pre-trained model or ready-made reinforcement-learning agent was used.